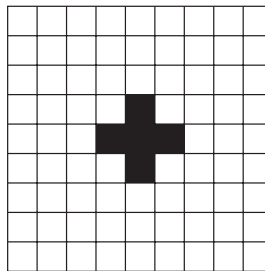
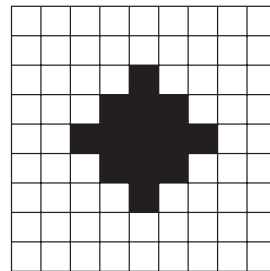
 $n = 0$  $n = 1$  $n = 2$ 

- 13. Page numbering** Find the total number of decimal digits needed for numbering pages in a book of 1000 pages. Assume that the pages are numbered consecutively starting with 1.

2.4 Mathematical Analysis of Recursive Algorithms

In this section, we will see how to apply the general framework for analysis of algorithms to recursive algorithms. We start with an example often used to introduce novices to the idea of a recursive algorithm.

EXAMPLE 1 Compute the factorial function $F(n) = n!$ for an arbitrary nonnegative integer n . Since

$$n! = 1 \cdot \dots \cdot (n-1) \cdot n = (n-1)! \cdot n \quad \text{for } n \geq 1$$

and $0! = 1$ by definition, we can compute $F(n) = F(n-1) \cdot n$ with the following recursive algorithm.

ALGORITHM $F(n)$

```
//Computes  $n!$  recursively
//Input: A nonnegative integer  $n$ 
//Output: The value of  $n!$ 
if  $n = 0$  return 1
else return  $F(n-1) * n$ 
```

For simplicity, we consider n itself as an indicator of this algorithm's input size (rather than the number of bits in its binary expansion). The basic operation of the algorithm is multiplication,⁵ whose number of executions we denote $M(n)$. Since the function $F(n)$ is computed according to the formula

$$F(n) = F(n-1) \cdot n \quad \text{for } n > 0,$$

5. Alternatively, we could count the number of times the comparison $n = 0$ is executed, which is the same as counting the total number of calls made by the algorithm (see Problem 2 in this section's exercises).

the number of multiplications $M(n)$ needed to compute it must satisfy the equality

$$M(n) = \underset{\substack{\text{to compute} \\ F(n-1)}}{M(n-1)} + \underset{\substack{\text{to multiply} \\ F(n-1) \text{ by } n}}{1} \quad \text{for } n > 0.$$

Indeed, $M(n-1)$ multiplications are spent to compute $F(n-1)$, and one more multiplication is needed to multiply the result by n .

The last equation defines the sequence $M(n)$ that we need to find. This equation defines $M(n)$ not explicitly, i.e., as a function of n , but implicitly as a function of its value at another point, namely $n-1$. Such equations are called **recurrence relations** or, for brevity, **recurrences**. Recurrence relations play an important role not only in analysis of algorithms but also in some areas of applied mathematics. They are usually studied in detail in courses on discrete mathematics or discrete structures; a very brief tutorial on them is provided in Appendix B. Our goal now is to solve the recurrence relation $M(n) = M(n-1) + 1$, i.e., to find an explicit formula for $M(n)$ in terms of n only.

Note, however, that there is not one but infinitely many sequences that satisfy this recurrence. (Can you give examples of, say, two of them?) To determine a solution uniquely, we need an **initial condition** that tells us the value with which the sequence starts. We can obtain this value by inspecting the condition that makes the algorithm stop its recursive calls:

if $n = 0$ return 1.

This tells us two things. First, since the calls stop when $n = 0$, the smallest value of n for which this algorithm is executed and hence $M(n)$ defined is 0. Second, by inspecting the pseudocode's exiting line, we can see that when $n = 0$, the algorithm performs no multiplications. Therefore, the initial condition we are after is

$$M(0) = 0.$$

↑

↑

the calls stop when $n = 0$ no multiplications when $n = 0$

Thus, we succeeded in setting up the recurrence relation and initial condition for the algorithm's number of multiplications $M(n)$:

$$\begin{aligned} M(n) &= M(n-1) + 1 \quad \text{for } n > 0, \\ M(0) &= 0. \end{aligned} \tag{2.2}$$

Before we embark on a discussion of how to solve this recurrence, let us pause to reiterate an important point. We are dealing here with two recursively defined functions. The first is the factorial function $F(n)$ itself; it is defined by the recurrence

$$\begin{aligned} F(n) &= F(n-1) \cdot n \quad \text{for every } n > 0, \\ F(0) &= 1. \end{aligned}$$

The second is the number of multiplications $M(n)$ needed to compute $F(n)$ by the recursive algorithm whose pseudocode was given at the beginning of the section.

As we just showed, $M(n)$ is defined by recurrence (2.2). And it is recurrence (2.2) that we need to solve now.

Though it is not difficult to “guess” the solution here (what sequence starts with 0 when $n = 0$ and increases by 1 on each step?), it will be more useful to arrive at it in a systematic fashion. From the several techniques available for solving recurrence relations, we use what can be called the **method of backward substitutions**. The method’s idea (and the reason for the name) is immediately clear from the way it applies to solving our particular recurrence:

$$\begin{aligned} M(n) &= M(n-1) + 1 && \text{substitute } M(n-1) = M(n-2) + 1 \\ &= [M(n-2) + 1] + 1 = M(n-2) + 2 && \text{substitute } M(n-2) = M(n-3) + 1 \\ &= [M(n-3) + 1] + 2 = M(n-3) + 3. \end{aligned}$$

After inspecting the first three lines, we see an emerging pattern, which makes it possible to predict not only the next line (what would it be?) but also a general formula for the pattern: $M(n) = M(n-i) + i$. Strictly speaking, the correctness of this formula should be proved by mathematical induction, but it is easier to get to the solution as follows and then verify its correctness.

What remains to be done is to take advantage of the initial condition given. Since it is specified for $n = 0$, we have to substitute $i = n$ in the pattern’s formula to get the ultimate result of our backward substitutions:

$$M(n) = M(n-1) + 1 = \cdots = M(n-i) + i = \cdots = M(n-n) + n = n.$$

You should not be disappointed after exerting so much effort to get this “obvious” answer. The benefits of the method illustrated in this simple example will become clear very soon, when we have to solve more difficult recurrences. Also, note that the simple iterative algorithm that accumulates the product of n consecutive integers requires the same number of multiplications, and it does so without the overhead of time and space used for maintaining the recursion’s stack.

The issue of time efficiency is actually not that important for the problem of computing $n!$, however. As we saw in Section 2.1, the function’s values get so large so fast that we can realistically compute exact values of $n!$ only for very small n ’s. Again, we use this example just as a simple and convenient vehicle to introduce the standard approach to analyzing recursive algorithms. ■

Generalizing our experience with investigating the recursive algorithm for computing $n!$, we can now outline a general plan for investigating recursive algorithms.

General Plan for Analyzing the Time Efficiency of Recursive Algorithms

1. Decide on a parameter (or parameters) indicating an input’s size.
2. Identify the algorithm’s basic operation.

3. Check whether the number of times the basic operation is executed can vary on different inputs of the same size; if it can, the worst-case, average-case, and best-case efficiencies must be investigated separately.
4. Set up a recurrence relation, with an appropriate initial condition, for the number of times the basic operation is executed.
5. Solve the recurrence or, at least, ascertain the order of growth of its solution.

EXAMPLE 2 As our next example, we consider another educational workhorse of recursive algorithms: the ***Tower of Hanoi*** puzzle. In this puzzle, we (or mythical monks, if you do not like to move disks) have n disks of different sizes that can slide onto any of three pegs. Initially, all the disks are on the first peg in order of size, the largest on the bottom and the smallest on top. The goal is to move all the disks to the third peg, using the second one as an auxiliary, if necessary. We can move only one disk at a time, and it is forbidden to place a larger disk on top of a smaller one.

The problem has an elegant recursive solution, which is illustrated in Figure 2.4. To move $n > 1$ disks from peg 1 to peg 3 (with peg 2 as auxiliary), we first move recursively $n - 1$ disks from peg 1 to peg 2 (with peg 3 as auxiliary), then move the largest disk directly from peg 1 to peg 3, and, finally, move recursively $n - 1$ disks from peg 2 to peg 3 (using peg 1 as auxiliary). Of course, if $n = 1$, we simply move the single disk directly from the source peg to the destination peg.

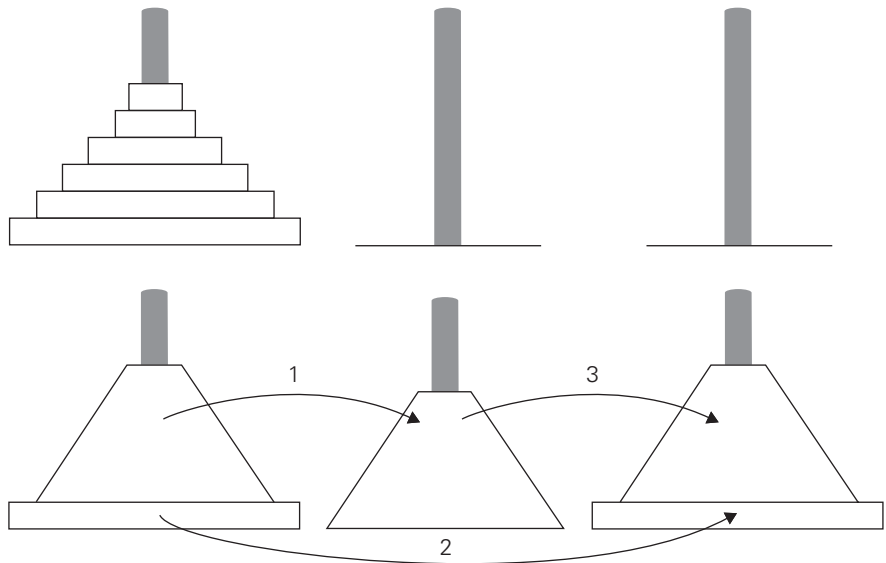


FIGURE 2.4 Recursive solution to the Tower of Hanoi puzzle.

Let us apply the general plan outlined above to the Tower of Hanoi problem. The number of disks n is the obvious choice for the input's size indicator, and so is moving one disk as the algorithm's basic operation. Clearly, the number of moves $M(n)$ depends on n only, and we get the following recurrence equation for it:

$$M(n) = M(n-1) + 1 + M(n-1) \quad \text{for } n > 1.$$

With the obvious initial condition $M(1) = 1$, we have the following recurrence relation for the number of moves $M(n)$:

$$\begin{aligned} M(n) &= 2M(n-1) + 1 \quad \text{for } n > 1, \\ M(1) &= 1. \end{aligned} \tag{2.3}$$

We solve this recurrence by the same method of backward substitutions:

$$\begin{aligned} M(n) &= 2M(n-1) + 1 && \text{sub. } M(n-1) = 2M(n-2) + 1 \\ &= 2[2M(n-2) + 1] + 1 = 2^2M(n-2) + 2 + 1 && \text{sub. } M(n-2) = 2M(n-3) + 1 \\ &= 2^2[2M(n-3) + 1] + 2 + 1 = 2^3M(n-3) + 2^2 + 2 + 1. \end{aligned}$$

The pattern of the first three sums on the left suggests that the next one will be $2^4M(n-4) + 2^3 + 2^2 + 2 + 1$, and generally, after i substitutions, we get

$$M(n) = 2^i M(n-i) + 2^{i-1} + 2^{i-2} + \cdots + 2 + 1 = 2^i M(n-i) + 2^i - 1.$$

Since the initial condition is specified for $n = 1$, which is achieved for $i = n - 1$, we get the following formula for the solution to recurrence (2.3):

$$\begin{aligned} M(n) &= 2^{n-1}M(n - (n-1)) + 2^{n-1} - 1 \\ &= 2^{n-1}M(1) + 2^{n-1} - 1 = 2^{n-1} + 2^{n-1} - 1 = 2^n - 1. \end{aligned}$$

Thus, we have an exponential algorithm, which will run for an unimaginably long time even for moderate values of n (see Problem 5 in this section's exercises). This is not due to the fact that this particular algorithm is poor; in fact, it is not difficult to prove that this is the most efficient algorithm possible for this problem. It is the problem's intrinsic difficulty that makes it so computationally hard. Still, this example makes an important general point:

One should be careful with recursive algorithms because their succinctness may mask their inefficiency.

When a recursive algorithm makes more than a single call to itself, it can be useful for analysis purposes to construct a tree of its recursive calls. In this tree, nodes correspond to recursive calls, and we can label them with the value of the parameter (or, more generally, parameters) of the calls. For the Tower of Hanoi example, the tree is given in Figure 2.5. By counting the number of nodes in the tree, we can get the total number of calls made by the Tower of Hanoi algorithm:

$$C(n) = \sum_{l=0}^{n-1} 2^l \quad (\text{where } l \text{ is the level in the tree in Figure 2.5}) = 2^n - 1.$$

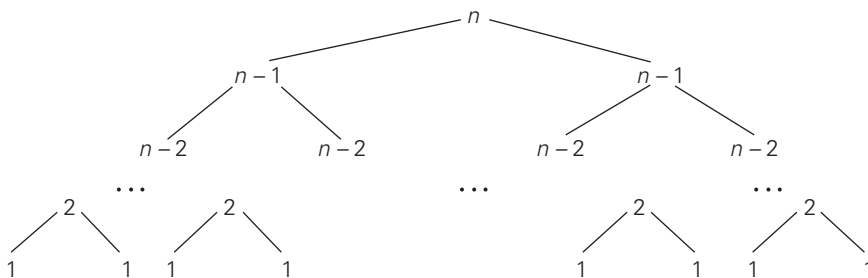


FIGURE 2.5 Tree of recursive calls made by the recursive algorithm for the Tower of Hanoi puzzle.

The number agrees, as it should, with the move count obtained earlier. ■

EXAMPLE 3 As our next example, we investigate a recursive version of the algorithm discussed at the end of Section 2.3.

ALGORITHM *BinRec*(n)

//Input: A positive decimal integer n

//Output: The number of binary digits in n 's binary representation

if $n = 1$ **return** 1

else return $\text{BinRec}(\lfloor n/2 \rfloor) + 1$

Let us set up a recurrence and an initial condition for the number of additions $A(n)$ made by the algorithm. The number of additions made in computing $\text{BinRec}(\lfloor n/2 \rfloor)$ is $A(\lfloor n/2 \rfloor)$, plus one more addition is made by the algorithm to increase the returned value by 1. This leads to the recurrence

$$A(n) = A(\lfloor n/2 \rfloor) + 1 \quad \text{for } n > 1. \quad (2.4)$$

Since the recursive calls end when n is equal to 1 and there are no additions made then, the initial condition is

$$A(1) = 0.$$

The presence of $\lfloor n/2 \rfloor$ in the function's argument makes the method of backward substitutions stumble on values of n that are not powers of 2. Therefore, the standard approach to solving such a recurrence is to solve it only for $n = 2^k$ and then take advantage of the theorem called the **smoothness rule** (see Appendix B), which claims that under very broad assumptions the order of growth observed for $n = 2^k$ gives a correct answer about the order of growth for all values of n . (Alternatively, after getting a solution for powers of 2, we can sometimes fine-tune this solution to get a formula valid for an arbitrary n .) So let us apply this recipe to our recurrence, which for $n = 2^k$ takes the form

$$A(2^k) = A(2^{k-1}) + 1 \quad \text{for } k > 0,$$

$$A(2^0) = 0.$$

Now backward substitutions encounter no problems:

$$\begin{aligned}
 A(2^k) &= A(2^{k-1}) + 1 && \text{substitute } A(2^{k-1}) = A(2^{k-2}) + 1 \\
 &= [A(2^{k-2}) + 1] + 1 = A(2^{k-2}) + 2 && \text{substitute } A(2^{k-2}) = A(2^{k-3}) + 1 \\
 &= [A(2^{k-3}) + 1] + 2 = A(2^{k-3}) + 3 && \dots \\
 &\dots && \\
 &= A(2^{k-i}) + i && \\
 &\dots && \\
 &= A(2^{k-k}) + k.
 \end{aligned}$$

Thus, we end up with

$$A(2^k) = A(1) + k = k,$$

or, after returning to the original variable $n = 2^k$ and hence $k = \log_2 n$,

$$A(n) = \log_2 n \in \Theta(\log n).$$

In fact, one can prove (Problem 7 in this section's exercises) that the exact solution for an arbitrary value of n is given by just a slightly more refined formula $A(n) = \lfloor \log_2 n \rfloor$. ■

This section provides an introduction to the analysis of recursive algorithms. These techniques will be used throughout the book and expanded further as necessary. In the next section, we discuss the Fibonacci numbers; their analysis involves more difficult recurrence relations to be solved by a method different from backward substitutions.

Exercises 2.4

1. Solve the following recurrence relations.
 - a. $x(n) = x(n-1) + 5$ for $n > 1$, $x(1) = 0$
 - b. $x(n) = 3x(n-1)$ for $n > 1$, $x(1) = 4$
 - c. $x(n) = x(n-1) + n$ for $n > 0$, $x(0) = 0$
 - d. $x(n) = x(n/2) + n$ for $n > 1$, $x(1) = 1$ (solve for $n = 2^k$)
 - e. $x(n) = x(n/3) + 1$ for $n > 1$, $x(1) = 1$ (solve for $n = 3^k$)
2. Set up and solve a recurrence relation for the number of calls made by $F(n)$, the recursive algorithm for computing $n!$.
3. Consider the following recursive algorithm for computing the sum of the first n cubes: $S(n) = 1^3 + 2^3 + \dots + n^3$.

ALGORITHM $S(n)$

```
//Input: A positive integer  $n$ 
//Output: The sum of the first  $n$  cubes
if  $n = 1$  return 1
else return  $S(n - 1) + n * n * n$ 
```

- a. Set up and solve a recurrence relation for the number of times the algorithm's basic operation is executed.
 - b. How does this algorithm compare with the straightforward nonrecursive algorithm for computing this sum?
4. Consider the following recursive algorithm.

ALGORITHM $Q(n)$

```
//Input: A positive integer  $n$ 
if  $n = 1$  return 1
else return  $Q(n - 1) + 2 * n - 1$ 
```

- a. Set up a recurrence relation for this function's values and solve it to determine what this algorithm computes.
- b. Set up a recurrence relation for the number of multiplications made by this algorithm and solve it.
- c. Set up a recurrence relation for the number of additions/subtractions made by this algorithm and solve it.

**5. Tower of Hanoi**

- a. In the original version of the Tower of Hanoi puzzle, as it was published in the 1890s by Édouard Lucas, a French mathematician, the world will end after 64 disks have been moved from a mystical Tower of Brahma. Estimate the number of years it will take if monks could move one disk per minute. (Assume that monks do not eat, sleep, or die.)
- b. How many moves are made by the i th largest disk ($1 \leq i \leq n$) in this algorithm?
- c. Find a nonrecursive algorithm for the Tower of Hanoi puzzle and implement it in the language of your choice.



6. *Restricted Tower of Hanoi* Consider the version of the Tower of Hanoi puzzle in which n disks have to be moved from peg A to peg C using peg B so that any move should either place a disk on peg B or move a disk from that peg. (Of course, the prohibition of placing a larger disk on top of a smaller one remains in place, too.) Design a recursive algorithm for this problem and find the number of moves made by it.

7.
 - a. Prove that the exact number of additions made by the recursive algorithm $\text{BinRec}(n)$ for an arbitrary positive decimal integer n is $\lfloor \log_2 n \rfloor$.
 - b. Set up a recurrence relation for the number of additions made by the nonrecursive version of this algorithm (see Section 2.3, Example 4) and solve it.
8.
 - a. Design a recursive algorithm for computing 2^n for any nonnegative integer n that is based on the formula $2^n = 2^{n-1} + 2^{n-1}$.
 - b. Set up a recurrence relation for the number of additions made by the algorithm and solve it.
 - c. Draw a tree of recursive calls for this algorithm and count the number of calls made by the algorithm.
 - d. Is it a good algorithm for solving this problem?
9. Consider the following recursive algorithm.

ALGORITHM $\text{Riddle}(A[0..n-1])$
 //Input: An array $A[0..n-1]$ of real numbers
if $n = 1$ **return** $A[0]$
else $\text{temp} \leftarrow \text{Riddle}(A[0..n-2])$
 if $\text{temp} \leq A[n-1]$ **return** temp
 else return $A[n-1]$

- a. What does this algorithm compute?
 - b. Set up a recurrence relation for the algorithm's basic operation count and solve it.
10. Consider the following algorithm to check whether a graph defined by its adjacency matrix is complete.

ALGORITHM $\text{GraphComplete}(A[0..n-1, 0..n-1])$
 //Input: Adjacency matrix $A[0..n-1, 0..n-1]$ of an undirected graph G
 //Output: 1 (true) if G is complete and 0 (false) otherwise
if $n = 1$ **return** 1 //one-vertex graph is complete by definition
else
 if not $\text{GraphComplete}(A[0..n-2, 0..n-2])$ **return** 0
 else for $j \leftarrow 0$ **to** $n-2$ **do**
 if $A[n-1, j] = 0$ **return** 0
 return 1

What is the algorithm's efficiency class in the worst case?

11. The determinant of an $n \times n$ matrix

$$A = \begin{bmatrix} a_{00} & \cdots & a_{0\,n-1} \\ a_{10} & \cdots & a_{1\,n-1} \\ \vdots & & \vdots \\ a_{n-1\,0} & \cdots & a_{n-1\,n-1} \end{bmatrix},$$

denoted $\det A$, can be defined as a_{00} for $n = 1$ and, for $n > 1$, by the recursive formula

$$\det A = \sum_{j=0}^{n-1} s_j a_{0j} \det A_j,$$

where s_j is $+1$ if j is even and -1 if j is odd, a_{0j} is the element in row 0 and column j , and A_j is the $(n-1) \times (n-1)$ matrix obtained from matrix A by deleting its row 0 and column j .

- a. Set up a recurrence relation for the number of multiplications made by the algorithm implementing this recursive definition.
- b. Without solving the recurrence, what can you say about the solution's order of growth as compared to $n!$?



12. *von Neumann's neighborhood revisited* Find the number of cells in the von Neumann neighborhood of range n (Problem 12 in Exercises 2.3) by setting up and solving a recurrence relation.



13. *Frying hamburgers* There are n hamburgers to be fried on a small grill that can hold only two hamburgers at a time. Each hamburger has to be fried on both sides; frying one side of a hamburger takes 1 minute, regardless of whether one or two hamburgers are fried at the same time. Consider the following recursive algorithm for executing this task in the minimum amount of time. If $n \leq 2$, fry the hamburger or the two hamburgers together on each side. If $n > 2$, fry any two hamburgers together on each side and then apply the same procedure recursively to the remaining $n - 2$ hamburgers.
 - a. Set up and solve the recurrence for the amount of time this algorithm needs to fry n hamburgers.
 - b. Explain why this algorithm does *not* fry the hamburgers in the minimum amount of time for all $n > 0$.
 - c. Give a correct recursive algorithm that executes the task in the minimum amount of time.



14. *Celebrity problem* A celebrity among a group of n people is a person who knows nobody but is known by everybody else. The task is to identify a celebrity by only asking questions to people of the form "Do you know him/her?" Design an efficient algorithm to identify a celebrity or determine that the group has no such person. How many questions does your algorithm need in the worst case?

2.5 Example: Computing the n th Fibonacci Number

In this section, we consider the **Fibonacci numbers**, a famous sequence

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots \quad (2.5)$$

that can be defined by the simple recurrence

$$F(n) = F(n-1) + F(n-2) \quad \text{for } n > 1 \quad (2.6)$$

and two initial conditions

$$F(0) = 0, \quad F(1) = 1. \quad (2.7)$$

The Fibonacci numbers were introduced by Leonardo Fibonacci in 1202 as a solution to a problem about the size of a rabbit population (Problem 2 in this section's exercises). Many more examples of Fibonacci-like numbers have since been discovered in the natural world, and they have even been used in predicting the prices of stocks and commodities. There are some interesting applications of the Fibonacci numbers in computer science as well. For example, worst-case inputs for Euclid's algorithm discussed in Section 1.1 happen to be consecutive elements of the Fibonacci sequence. In this section, we briefly consider algorithms for computing the n th element of this sequence. Among other benefits, the discussion will provide us with an opportunity to introduce another method for solving recurrence relations useful for analysis of recursive algorithms.

To start, let us get an explicit formula for $F(n)$. If we try to apply the method of backward substitutions to solve recurrence (2.6), we will fail to get an easily discernible pattern. Instead, we can take advantage of a theorem that describes solutions to a **homogeneous second-order linear recurrence with constant coefficients**

$$ax(n) + bx(n-1) + cx(n-2) = 0, \quad (2.8)$$

where a , b , and c are some fixed real numbers ($a \neq 0$) called the coefficients of the recurrence and $x(n)$ is the generic term of an unknown sequence to be found. Applying this theorem to our recurrence with the initial conditions given—see Appendix B—we obtain the formula

$$F(n) = \frac{1}{\sqrt{5}}(\phi^n - \hat{\phi}^n), \quad (2.9)$$

where $\phi = (1 + \sqrt{5})/2 \approx 1.61803$ and $\hat{\phi} = -1/\phi \approx -0.61803$.⁶ It is hard to believe that formula (2.9), which includes arbitrary integer powers of irrational numbers, yields nothing else but all the elements of Fibonacci sequence (2.5), but it does!

One of the benefits of formula (2.9) is that it immediately implies that $F(n)$ grows exponentially (remember Fibonacci's rabbits?), i.e., $F(n) \in \Theta(\phi^n)$. This

6. Constant ϕ is known as the **golden ratio**. Since antiquity, it has been considered the most pleasing ratio of a rectangle's two sides to the human eye and might have been consciously used by ancient architects and sculptors.

follows from the observation that $\hat{\phi}$ is a fraction between -1 and 0 , and hence $\hat{\phi}^n$ gets infinitely small as n goes to infinity. In fact, one can prove that the impact of the second term $\frac{1}{\sqrt{5}}\hat{\phi}^n$ on the value of $F(n)$ can be obtained by rounding off the value of the first term to the nearest integer. In other words, for every nonnegative integer n ,

$$F(n) = \frac{1}{\sqrt{5}}\phi^n \quad \text{rounded to the nearest integer.} \quad (2.10)$$

In the algorithms that follow, we consider, for the sake of simplicity, such operations as additions and multiplications at unit cost. Since the Fibonacci numbers grow infinitely large (and grow very rapidly), a more detailed analysis than the one offered here is warranted. In fact, it is the size of the numbers rather than a time-efficient method for computing them that should be of primary concern here. Still, these caveats notwithstanding, the algorithms we outline and their analysis provide useful examples for a student of the design and analysis of algorithms.

To begin with, we can use recurrence (2.6) and initial conditions (2.7) for the obvious recursive algorithm for computing $F(n)$.

ALGORITHM $F(n)$

```
//Computes the  $n$ th Fibonacci number recursively by using its definition
//Input: A nonnegative integer  $n$ 
//Output: The  $n$ th Fibonacci number
if  $n \leq 1$  return  $n$ 
else return  $F(n - 1) + F(n - 2)$ 
```

Before embarking on its formal analysis, can you tell whether this is an efficient algorithm? Well, we need to do a formal analysis anyway. The algorithm's basic operation is clearly addition, so let $A(n)$ be the number of additions performed by the algorithm in computing $F(n)$. Then the numbers of additions needed for computing $F(n - 1)$ and $F(n - 2)$ are $A(n - 1)$ and $A(n - 2)$, respectively, and the algorithm needs one more addition to compute their sum. Thus, we get the following recurrence for $A(n)$:

$$\begin{aligned} A(n) &= A(n - 1) + A(n - 2) + 1 \quad \text{for } n > 1, \\ A(0) &= 0, \quad A(1) = 0. \end{aligned} \quad (2.11)$$

The recurrence $A(n) - A(n - 1) - A(n - 2) = 1$ is quite similar to recurrence $F(n) - F(n - 1) - F(n - 2) = 0$, but its right-hand side is not equal to zero. Such recurrences are called **inhomogeneous**. There are general techniques for solving inhomogeneous recurrences (see Appendix B or any textbook on discrete mathematics), but for this particular recurrence, a special trick leads to a faster solution. We can reduce our inhomogeneous recurrence to a homogeneous one by rewriting it as

$$[A(n) + 1] - [A(n - 1) + 1] - [A(n - 2) + 1] = 0$$

and substituting $B(n) = A(n) + 1$:

$$\begin{aligned} B(n) - B(n-1) - B(n-2) &= 0, \\ B(0) &= 1, \quad B(1) = 1. \end{aligned}$$

This homogeneous recurrence can be solved exactly in the same manner as recurrence (2.6) was solved to find an explicit formula for $F(n)$. But it can actually be avoided by noting that $B(n)$ is, in fact, the same recurrence as $F(n)$ except that it starts with two 1's and thus runs one step ahead of $F(n)$. So $B(n) = F(n+1)$, and

$$A(n) = B(n) - 1 = F(n+1) - 1 = \frac{1}{\sqrt{5}}(\phi^{n+1} - \hat{\phi}^{n+1}) - 1.$$

Hence, $A(n) \in \Theta(\phi^n)$, and if we measure the size of n by the number of bits $b = \lfloor \log_2 n \rfloor + 1$ in its binary representation, the efficiency class will be even worse, namely, doubly exponential: $A(b) \in \Theta(\phi^{2^b})$.

The poor efficiency class of the algorithm could be anticipated by the nature of recurrence (2.11). Indeed, it contains two recursive calls with the sizes of smaller instances only slightly smaller than size n . (Have you encountered such a situation before?) We can also see the reason behind the algorithm's inefficiency by looking at a recursive tree of calls tracing the algorithm's execution. An example of such a tree for $n = 5$ is given in Figure 2.6. Note that the same values of the function are being evaluated here again and again, which is clearly extremely inefficient.

We can obtain a much faster algorithm by simply computing the successive elements of the Fibonacci sequence iteratively, as is done in the following algorithm.

ALGORITHM *Fib*(n)

```
//Computes the  $n$ th Fibonacci number iteratively by using its definition
//Input: A nonnegative integer  $n$ 
//Output: The  $n$ th Fibonacci number
 $F[0] \leftarrow 0$ ;  $F[1] \leftarrow 1$ 
for  $i \leftarrow 2$  to  $n$  do
     $F[i] \leftarrow F[i-1] + F[i-2]$ 
return  $F[n]$ 
```

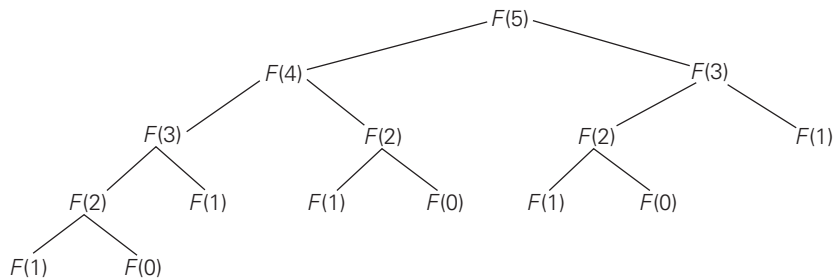


FIGURE 2.6 Tree of recursive calls for computing the 5th Fibonacci number by the definition-based algorithm.

This algorithm clearly makes $n - 1$ additions. Hence, it is linear as a function of n and “only” exponential as a function of the number of bits b in n ’s binary representation. Note that using an extra array for storing all the preceding elements of the Fibonacci sequence can be avoided: storing just two values is necessary to accomplish the task (see Problem 8 in this section’s exercises).

The third alternative for computing the n th Fibonacci number lies in using formula (2.10). The efficiency of the algorithm will obviously be determined by the efficiency of an exponentiation algorithm used for computing ϕ^n . If it is done by simply multiplying ϕ by itself $n - 1$ times, the algorithm will be in $\Theta(n) = \Theta(2^b)$. There are faster algorithms for the exponentiation problem. For example, we will discuss $\Theta(\log n) = \Theta(b)$ algorithms for this problem in Chapters 4 and 6. Note also that special care should be exercised in implementing this approach to computing the n th Fibonacci number. Since all its intermediate results are irrational numbers, we would have to make sure that their approximations in the computer are accurate enough so that the final round-off yields a correct result.

Finally, there exists a $\Theta(\log n)$ algorithm for computing the n th Fibonacci number that manipulates only integers. It is based on the equality

$$\begin{bmatrix} F(n-1) & F(n) \\ F(n) & F(n+1) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^n \quad \text{for } n \geq 1$$

and an efficient way of computing matrix powers.

Exercises 2.5

1. Find a Web site dedicated to applications of the Fibonacci numbers and study it.



2. *Fibonacci’s rabbits problem* A man put a pair of rabbits in a place surrounded by a wall. How many pairs of rabbits will be there in a year if the initial pair of rabbits (male and female) are newborn and all rabbit pairs are not fertile during their first month of life but thereafter give birth to one new male/female pair at the end of every month?



3. *Climbing stairs* Find the number of different ways to climb an n -stair staircase if each step is either one or two stairs. For example, a 3-stair staircase can be climbed three ways: 1-1-1, 1-2, and 2-1.
4. How many even numbers are there among the first n Fibonacci numbers, i.e., among the numbers $F(0)$, $F(1)$, $\dots$, $F(n-1)$? Give a closed-form formula valid for every $n > 0$.
5. Check by direct substitutions that the function $\frac{1}{\sqrt{5}}(\phi^n - \hat{\phi}^n)$ indeed satisfies recurrence (2.6) and initial conditions (2.7).
6. The maximum values of the Java primitive types `int` and `long` are $2^{31} - 1$ and $2^{63} - 1$, respectively. Find the smallest n for which the n th Fibonacci number is not going to fit in a memory allocated for

- a. the type `int`. b. the type `long`.
7. Consider the recursive definition-based algorithm for computing the n th Fibonacci number $F(n)$. Let $C(n)$ and $Z(n)$ be the number of times $F(1)$ and $F(0)$ are computed, respectively. Prove that
- a. $C(n) = F(n)$. b. $Z(n) = F(n - 1)$.
8. Improve algorithm *Fib* of the text so that it requires only $\Theta(1)$ space.
9. Prove the equality

$$\begin{bmatrix} F(n-1) & F(n) \\ F(n) & F(n+1) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^n \quad \text{for } n \geq 1.$$

10. How many modulo divisions are made by Euclid's algorithm on two consecutive Fibonacci numbers $F(n)$ and $F(n - 1)$ as the algorithm's input?
11. *Dissecting a Fibonacci rectangle* Given a rectangle whose sides are two consecutive Fibonacci numbers, design an algorithm to dissect it into squares with no more than two squares being the same size. What is the time efficiency class of your algorithm?
12. In the language of your choice, implement two algorithms for computing the last five digits of the n th Fibonacci number that are based on (a) the recursive definition-based algorithm $F(n)$; (b) the iterative definition-based algorithm *Fib*(n). Perform an experiment to find the largest value of n for which your programs run under 1 minute on your computer.



2.6 Empirical Analysis of Algorithms

In Sections 2.3 and 2.4, we saw how algorithms, both nonrecursive and recursive, can be analyzed mathematically. Though these techniques can be applied successfully to many simple algorithms, the power of mathematics, even when enhanced with more advanced techniques (see [Sed96], [Pur04], [Gra94], and [Gre07]), is far from limitless. In fact, even some seemingly simple algorithms have proved to be very difficult to analyze with mathematical precision and certainty. As we pointed out in Section 2.1, this is especially true for the average-case analysis.

The principal alternative to the mathematical analysis of an algorithm's efficiency is its empirical analysis. This approach implies steps spelled out in the following plan.

General Plan for the Empirical Analysis of Algorithm Time Efficiency

1. Understand the experiment's purpose.
2. Decide on the efficiency metric M to be measured and the measurement unit (an operation count vs. a time unit).
3. Decide on characteristics of the input sample (its range, size, and so on).
4. Prepare a program implementing the algorithm (or algorithms) for the experimentation.